Distributed Gaming Analytics System

Using Ray for Parallel Player Analysis and Prediction

Project Report

Submitted By

Ananyo Sen
Anindya Midhey

Submitted To

Champak Kumar Dutta

Department of Computer Science
RKMVERI

Submission Date

April 30, 2026

Contents

Abstract

The increasing demand for real-time analytics in gaming environments requires systems capable of processing large volumes of player data efficiently. Traditional sequential systems become inefficient when handling multiple simultaneous requests due to increased response time and resource limitations.

This project presents a Distributed Gaming Analytics System that leverages the Ray distributed computing framework for parallel execution of player analysis and prediction tasks.

The system integrates machine learning models, statistical performance analysis, Flask-based backend services, and a distributed execution environment using Ray clusters.

Each player is treated as an independent task and distributed across worker nodes for simultaneous processing.

To improve performance, several optimization techniques including lazy initialization, lightweight object sharing, and local model loading were implemented.

Experimental observations demonstrate improved scalability, reduced processing time, and efficient utilization of system resources compared to sequential processing approaches.

Chapter 1

Introduction

The gaming industry has experienced rapid growth in recent years, leading to an increase in the generation of player performance data. Modern multiplayer games continuously generate large amounts of information related to player activities, statistics, and gameplay behavior. Metrics such as kills, assists, deaths, Average Damage per Round (ADR), KAST percentage, and Kill-Death difference are commonly used to evaluate individual player performance.

Gaming analytics plays an important role in understanding player behavior and extracting meaningful insights from collected game data. Such analytics are useful for performance evaluation, strategy development, player ranking systems, recommendation systems, and prediction of future player outcomes.

Traditional analytics systems generally operate using sequential processing methods, where tasks are executed one after another. While this approach may work for small datasets or limited users, it becomes inefficient when multiple users simultaneously request predictions or statistical analyses. As the number of users increases, sequential processing introduces delays, higher response time, and inefficient resource utilization.

Modern gaming platforms require near real-time processing capabilities because players expect immediate results and feedback. Large-scale gaming applications may need to process data for multiple players simultaneously. Therefore, there exists a need for systems capable of handling multiple independent tasks concurrently while maintaining scalability and performance.

Distributed computing addresses this limitation by dividing computational workloads across multiple processing nodes. Instead of executing tasks sequentially, independent tasks can be distributed among multiple workers and processed simultaneously.

This project presents a Distributed Gaming Analytics System using the Ray framework for distributed task execution. The system combines machine learning-based prediction, statistical analysis, Flask-based backend communication, and distributed processing to

improve overall efficiency and scalability.

The major components of the proposed system include:

- Machine learning models for prediction
- Statistical analysis for performance evaluation
- Flask backend for handling requests
- Ray framework for distributed task scheduling
- Worker nodes for parallel computation

The system converts individual player data into independent tasks and distributes them across multiple worker nodes, allowing simultaneous execution and improved response time.

Chapter 2

Problem Statement and Motivation

Modern gaming systems frequently operate in environments where thousands or even millions of users simultaneously interact with online services. Such systems continuously process player statistics, authentication requests, matchmaking services, and analytical computations. Handling these requests efficiently becomes increasingly challenging as the number of users grows.

Traditional systems generally rely on centralized or sequential processing methods in which tasks are handled one after another. Although such approaches may function adequately for small-scale applications, they become inefficient under heavy workloads. Increasing user requests lead to higher waiting time, delayed response generation, and underutilization of available computational resources.

2.1 Real-World Motivation: Diablo III Launch Failure

A notable example highlighting the importance of scalability occurred during the launch of the game *Diablo III* in 2012. During its release, millions of users attempted to log into the system simultaneously. Since the game relied heavily on centralized online services, the backend infrastructure became overwhelmed by the sudden increase in concurrent requests.

This resulted in the famous **Error 37** issue, where players repeatedly received service unavailable messages and were unable to access the game.

Major consequences of this event included:

- Login failures for a large number of users
- Extremely long waiting queues
- Increased response delays

- Negative user experience
- Scalability limitations of backend systems

Although the issue was not caused purely by sequential processing, it exposed an important limitation of centralized systems: inability to efficiently handle large numbers of independent requests simultaneously.

The gaming industry gradually addressed such challenges by shifting toward distributed architectures, scalable cloud services, load balancing mechanisms, and parallel request processing systems.

2.2 Problem in Gaming Analytics

Similar challenges also exist within gaming analytics systems. Consider a situation where multiple users simultaneously request analysis and prediction of player performance.

A sequential system would process each player individually:

$$Total\ Processing\ Time = T_1 + T_2 + T_3 + \cdots + T_n$$

where:

- $T_1, T_2, T_3, ..., T_n$ represent execution time for each player task

As the number of users and players increases, total execution time grows significantly.

This leads to several issues:

- Increased response time
- System bottlenecks
- Poor scalability
- Reduced user experience
- Inefficient resource utilization

2.3 Motivation for Proposed System

To overcome these limitations, there is a need for a system capable of distributing workloads among multiple processing nodes and executing independent tasks simultaneously.

Distributed computing offers an effective solution by enabling:

- Parallel task execution
- Better utilization of computational resources
- Reduced response time
- Improved scalability
- Efficient handling of concurrent requests

The proposed Distributed Gaming Analytics System was motivated by these requirements. By using the Ray framework, player-related computations can be converted into independent tasks and distributed among worker nodes for simultaneous execution.

This approach transforms the traditional sequential workflow into a scalable distributed architecture capable of handling increasing workloads efficiently.

Chapter 3

Objectives

The primary objective of this project is to develop a scalable gaming analytics platform capable of efficiently handling multiple player-related computations through distributed computing techniques.

The specific objectives are listed below:

1. To develop a gaming analytics system capable of processing multiple player requests simultaneously.
2. To implement distributed task execution using the Ray framework.
3. To reduce the limitations associated with sequential processing systems.
4. To design machine learning models capable of predicting player performance metrics.
5. To perform statistical analysis of player data for performance evaluation.
6. To develop a user-friendly web interface for interaction.
7. To optimize resource utilization and reduce response time.
8. To demonstrate parallel task scheduling using multiple worker nodes.
9. To create a scalable architecture that can support future expansion.

Chapter 4

Literature Review

Distributed computing has become increasingly important due to the growing demand for scalable systems capable of processing large amounts of data efficiently. Various frameworks have been developed to distribute computational tasks across multiple machines and improve overall system performance.

Several frameworks were studied before selecting an appropriate technology for the proposed system.

4.1 Apache Hadoop

Apache Hadoop is a distributed computing framework designed for large-scale data processing using the MapReduce programming model.

Major advantages include:

- Reliable distributed storage
- Fault tolerance
- Capability of handling very large datasets

However, Hadoop also has several limitations:

- High processing latency
- Less suitable for real-time applications
- Complex programming structure

Hadoop is primarily designed for batch processing rather than real-time task execution.

4.2 Apache Spark

Apache Spark was developed to overcome the performance limitations of Hadoop by introducing in-memory computation.

Advantages include:

- Faster computation compared to Hadoop
- Support for iterative processing
- Efficient memory usage
- Support for machine learning libraries

Limitations include:

- Complex deployment configuration
- Increased memory requirements
- Higher overhead for lightweight tasks

Although Spark provides high-speed processing, it may introduce unnecessary complexity for smaller task-level parallel applications.

4.3 Dask

Dask is a flexible Python library designed for parallel computing and distributed task scheduling.

Advantages include:

- Native Python integration
- Dynamic task scheduling
- Easy scaling capabilities

Limitations include:

- Smaller ecosystem compared to Spark
- Performance limitations for certain workloads
- Limited support for some distributed machine learning scenarios

4.4 Ray Framework

Ray is an open-source distributed computing framework specifically designed for scalable applications and parallel task execution.

Major advantages include:

- Lightweight implementation
- Native Python support
- Simple task scheduling through remote functions
- Efficient object store architecture
- Support for machine learning workloads
- Scalable multi-node execution

Unlike traditional frameworks, Ray enables developers to convert ordinary Python functions into distributed tasks using simple decorators and remote calls.

Example:

```
@ray.remote
def predict_player():
    pass
```

This approach significantly reduces implementation complexity.

4.5 Comparative Analysis

Framework	Strength	Limitation
Hadoop	Reliable large-scale storage	High latency for real-time tasks
Spark	Fast in-memory processing	Complex deployment
Dask	Python ecosystem support	Smaller distributed ecosystem
Ray	Simple parallel task execution	Requires node coordination

Table 4.1: Comparison of Distributed Computing Frameworks

4.6 Justification for Selecting Ray

After evaluating different frameworks, Ray was selected for the proposed system because it aligned closely with the project requirements.

The reasons for selecting Ray include:

- Support for lightweight task-level parallelism
- Easy integration with Python
- Efficient distributed execution model
- Reduced implementation complexity
- Suitable for machine learning applications
- Better support for independent task scheduling

Since player analysis and prediction tasks are independent of one another, Ray provides an effective mechanism for distributing workloads among multiple worker nodes.

Chapter 5

System Architecture

The proposed Distributed Gaming Analytics System follows a layered architecture in which individual components are responsible for specific functionalities. The architecture combines user interaction, backend processing, distributed task scheduling, machine learning prediction, and statistical analysis into a unified system.

The primary objective of the architecture is to enable efficient processing of player-related computations while reducing response time through distributed execution.

5.1 Overall System Architecture

Figure 5.1 illustrates the overall architecture of the proposed system.

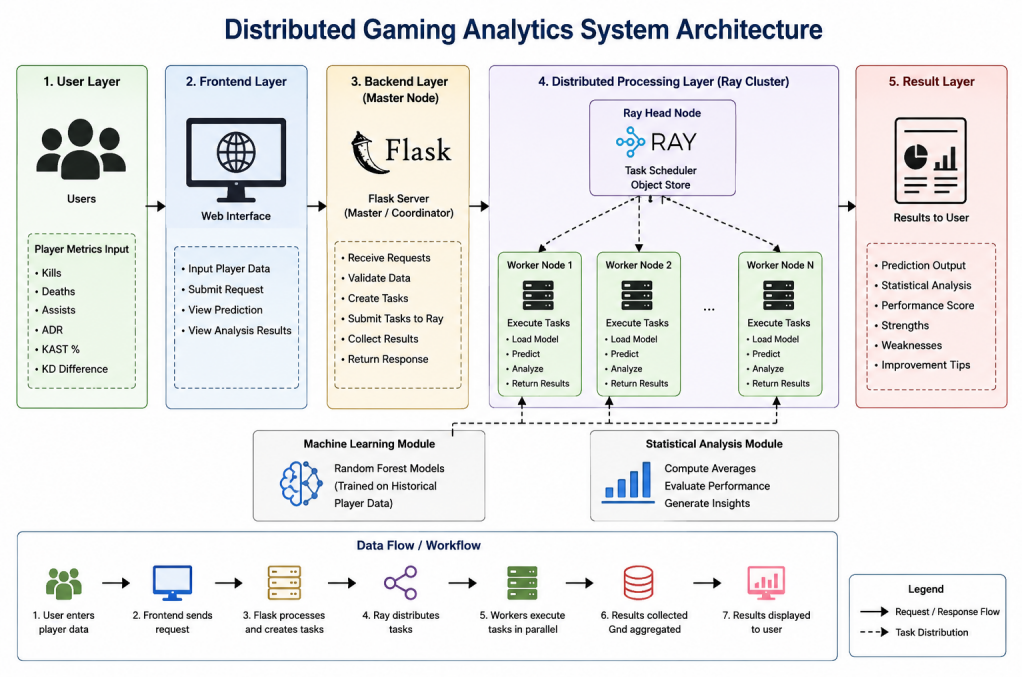

Figure 5.1: Overall System Architecture

The system architecture consists of five major layers:

1. User Layer
2. Frontend Layer
3. Backend Layer
4. Distributed Processing Layer
5. Result Layer

5.2 User Layer

The user layer represents the interaction point between the users and the system.

Users provide player statistics through a web interface. The input generally consists of performance metrics such as:

- Kills
- Deaths
- Assists
- ADR (Average Damage per Round)
- KAST Percentage
- Kill-Death Difference

The user initiates prediction or statistical analysis requests through the graphical interface.

5.3 Frontend Layer

The frontend layer is responsible for collecting information from users and providing a visual interface for interaction.

Functions of the frontend include:

- Receiving player data
- Sending requests to backend services
- Displaying prediction results

- Displaying statistical analysis output

HTML templates were used for interface implementation because of their simplicity and easy integration with Flask.

5.4 Backend Layer

The backend layer was implemented using Flask and serves as the communication bridge between the frontend interface and the distributed environment. It receives user requests, validates incoming player statistics, transforms the data into computational task objects, submits those tasks to Ray workers, and collects the generated outputs.

Within the proposed architecture, Flask also acts as the master node responsible for coordinating distributed execution and returning processed results to users.

5.5 Distributed Processing Layer

The distributed layer is the core component of the proposed system.

Ray was used to distribute computational tasks among multiple worker nodes.

The distributed execution process follows the following sequence:

1. Player data received by Flask
2. Data converted into independent tasks
3. Tasks submitted using remote functions
4. Ray scheduler distributes tasks
5. Worker nodes execute tasks
6. Results returned to Flask

This architecture allows multiple player-related tasks to execute simultaneously rather than sequentially.

5.6 Worker Nodes

Worker nodes perform the primary computational operations of the proposed system. Each worker independently loads trained machine learning models, performs statistical analysis, generates prediction outputs, and returns the results to the master node.

Independent execution among workers enables efficient utilization of available computational resources while reducing overall processing time.

5.7 Machine Learning Module

The machine learning module performs prediction tasks using historical player performance information. Random Forest models were selected because they combine multiple decision trees and provide robust prediction capability while reducing overfitting.

The selected model offers several advantages including improved prediction accuracy, support for nonlinear relationships among variables, and tolerance to noisy data. Trained model files are loaded locally within worker nodes to minimize communication overhead during distributed execution.

5.8 Statistical Analysis Module

The statistical analysis module evaluates player performance relative to dataset-wide statistics. The module computes average values, identifies strong and weak performance metrics, generates composite scores, and produces suggestions for improvement.

This analytical component enables users to understand player behavior beyond machine learning predictions and provides meaningful insights regarding overall performance.

5.9 Data Flow within the System

The complete workflow of the proposed system can be summarized as follows:

1. User enters player information
2. Frontend sends request to Flask backend
3. Flask converts input into task objects
4. Ray distributes tasks to workers
5. Workers execute prediction and analysis
6. Results are aggregated
7. Final output displayed to users

The use of distributed task execution significantly reduces response time compared to sequential processing.

Chapter 6

Methodology

This chapter explains the methodology adopted for developing the proposed Distributed Gaming Analytics System. The implementation combines machine learning, statistical analysis, Flask backend services, and distributed task execution using Ray.

The overall workflow of the system is illustrated through several stages beginning from data collection and ending with result generation.

6.1 Overall Methodology

The complete methodology follows the sequence below:

1. Data Collection and Preprocessing
2. Machine Learning Model Training
3. Statistical Analysis Development
4. Backend Development using Flask
5. Distributed Task Scheduling using Ray
6. Parallel Task Execution
7. Result Aggregation
8. User Output Generation

6.2 Data Collection and Preprocessing

Player statistics were obtained from the gaming dataset containing multiple performance-related attributes.

The dataset contains several player metrics such as:

- Kills
- Deaths
- Assists
- Average Damage per Round (ADR)
- KAST Percentage
- Kill-Death Difference

Before model training and analysis, preprocessing operations were performed to improve data quality.

Preprocessing steps included:

1. Removal of missing values
2. Elimination of invalid records
3. Selection of relevant features
4. Data transformation

The preprocessing stage helps improve prediction quality and reduces noise within the dataset.

6.3 Machine Learning Module

The machine learning module was developed to predict player-related statistics based on historical performance information.

Random Forest models were selected because of several advantages:

- High prediction accuracy
- Reduced overfitting
- Ability to handle nonlinear relationships
- Robustness against noisy data

The machine learning workflow consists of:

1. Loading dataset
2. Feature extraction
3. Splitting into training and testing data
4. Training Random Forest models
5. Saving trained models

The trained models are stored using serialized files and later loaded by worker nodes during distributed execution.

6.4 Statistical Analysis Module

Besides machine learning prediction, statistical analysis was also implemented for evaluating player performance.

Statistical computations involve comparison of player metrics with dataset averages.

The analysis module performs:

- Calculation of average values
- Identification of strong performance areas
- Identification of weak performance areas
- Performance score computation
- Generation of improvement suggestions

This module helps users understand player performance beyond predicted values.

6.5 Backend Development using Flask

Flask was selected for backend implementation because of its lightweight design and simple integration with Python libraries.

Within the proposed system, Flask acts as the communication layer between the frontend interface and the distributed environment. It receives user requests, validates incoming player information, transforms the received data into computational task objects, submits those tasks to Ray workers, collects outputs generated by worker nodes, and finally returns processed results through the user interface.

Flask additionally acts as the master node responsible for coordinating distributed execution.

6.6 Distributed Task Scheduling Using Ray

Ray enables distributed execution through remote functions.

The player-related computations are converted into distributed tasks.

The implementation uses the following approach:

```
futures = [
predict_player.remote(player,target)
for player in players
]
```

Each player becomes an independent computational task.

The Ray scheduler dynamically distributes these tasks among available worker nodes.

6.7 Parallel Task Execution

After tasks are assigned to workers, computations execute simultaneously.

Parallel execution follows:

1. Task creation
2. Task scheduling
3. Worker execution
4. Result generation
5. Result aggregation

Unlike sequential systems where tasks execute one after another, distributed execution processes multiple tasks simultaneously.

6.8 Result Aggregation

After task execution, results from workers are collected using:

```
results = ray.get(futures)
```

Ray waits until all workers finish execution and then aggregates outputs.

The aggregated results are then forwarded to Flask for rendering within the frontend interface.

6.9 Overall Working Procedure

The complete system workflow can be summarized as follows:

1. User enters player statistics
2. Flask receives request
3. Flask creates task objects
4. Ray distributes tasks
5. Workers execute tasks
6. Results collected
7. Results displayed to users

Chapter 7

Distributed Workflow and Optimization Techniques

The proposed system relies on distributed computing principles to improve performance and scalability. Instead of processing tasks sequentially, the system distributes independent player computations among multiple worker nodes.

The complete workflow and optimization techniques used in the system are explained in this chapter.

7.1 Distributed Workflow

The overall distributed workflow consists of multiple stages beginning from user interaction and ending with result generation.

Figure 7.1 illustrates the complete distributed workflow.

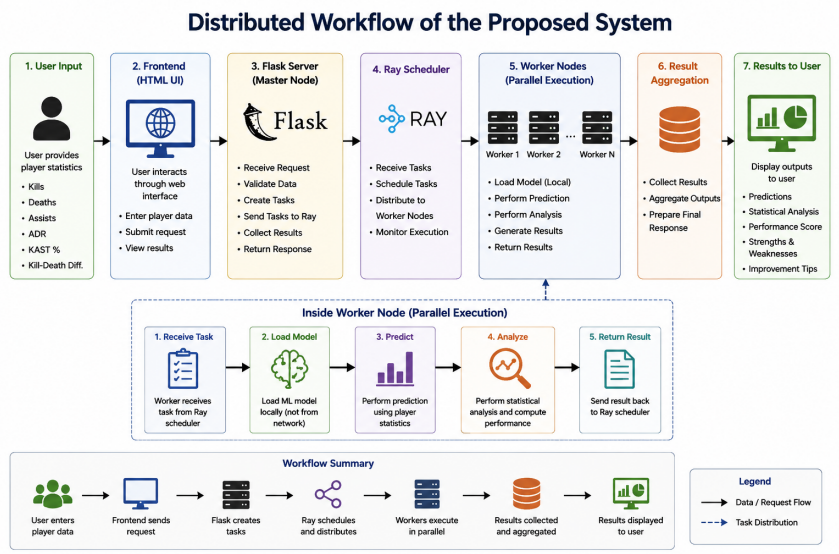

Figure 7.1: Distributed Workflow of Proposed System

7.2 Step 1: User Request Submission

The workflow begins when users enter player information through the frontend interface.

Input generally includes:

- Kills
- Deaths
- Assists
- ADR
- KAST Percentage
- Kill-Death Difference

The frontend sends this information to the Flask backend for processing.

7.3 Step 2: Task Preparation

After receiving the request, Flask validates the data and converts player information into structured task objects.

Each player record becomes an independent task.

This design allows tasks to execute separately without dependency on other player tasks.

7.4 Step 3: Task Distribution

Task distribution is performed using Ray remote functions.

The implementation uses:

```
futures = [
predict_player.remote(player,target)
for player in players
]
```

Explanation:

- `predict_player.remote()` converts a function into a distributed task
- Each player becomes an independent computation
- Ray dynamically assigns tasks to workers

7.5 Step 4: Parallel Worker Execution

After task assignment, worker nodes execute computations simultaneously.

Worker nodes perform:

- Loading machine learning models
- Statistical analysis
- Prediction generation
- Returning results

Unlike sequential execution, workers do not wait for each other.

Therefore:

$$Total\ Execution\ Time \approx max(T_1, T_2, T_3, \ldots, T_n)$$

instead of:

$$Total\ Execution\ Time = T_1 + T_2 + T_3 + \cdots + T_n$$

This significantly reduces response time.

7.6 Step 5: Result Aggregation

After execution, results are collected using:

```
results = ray.get(futures)
```

Explanation:

- Waits for all worker tasks to finish
- Collects task outputs
- Aggregates results
- Returns final response to Flask

7.7 Optimization Techniques

Several optimization strategies were implemented to reduce computation overhead and improve performance.

7.7.1 Lazy Initialization

Traditional systems often initialize all resources during application startup regardless of whether they are required immediately.

Such initialization increases startup time and memory consumption.

To avoid this issue, lazy initialization was implemented.

Code snippet:

```
if avgs_ref is None:
    init_ray_objects()
```

Explanation:

- Objects are initialized only when required
- Startup overhead is reduced
- Memory utilization becomes more efficient

7.7.2 Lightweight Object Sharing

Transferring complete datasets among workers creates unnecessary communication overhead.

Instead of transferring the entire dataset:

```
ray.put(dataset_averages)
```

was used.

Explanation:

- Only small average values are shared
- Reduces communication cost
- Improves worker efficiency
- Reduces memory consumption

7.7.3 Local Model Loading

Machine learning models were loaded locally inside worker nodes rather than transferring large model objects through the network.

Benefits include:

- Lower serialization cost

- Reduced network traffic
- Faster execution
- Lower memory overhead

7.7.4 Task-Level Parallelism

Task-level parallelism allows the system to convert each player into an independent computational task. Since the tasks do not depend on one another, multiple workers can process them simultaneously. This improves CPU utilization, reduces waiting time, and enables the architecture to scale efficiently as workload increases.

7.8 Advantages of Distributed Execution

The distributed approach provides several advantages:

- Reduced response time
- Efficient utilization of resources
- Scalability across multiple nodes
- Better handling of concurrent requests
- Improved overall system performance

Chapter 8

Experimental Setup and Results

This chapter describes the experimental environment used for implementing and evaluating the proposed Distributed Gaming Analytics System. The chapter also presents observations obtained from distributed execution.

8.1 Experimental Environment

The proposed system was implemented within a computer laboratory environment consisting of multiple computing nodes connected through a local network.

The experimental setup included:

- 3 independent clusters
- Each cluster contained:
  - 1 Master Node
  - 3 Worker Nodes
  - 1 Optional Worker Node
- Python virtual environment
- Ray distributed framework
- Flask backend services

8.2 Software Configuration

The software environment consisted of:

Component	Configuration
Programming Language	Python
Backend Framework	Flask
Distributed Framework	Ray
Machine Learning Library	Scikit-Learn
Data Processing Library	Pandas
Execution Environment	Virtual Environment (ray_env)

Table 8.1: Software Configuration

8.3 Cluster Initialization Process

The distributed environment was initialized using the following procedure:

1. Activate virtual environment

```
source ray_env/bin/activate
```

2. Start master node

```
ray start --head --port=6379
```

3. Connect worker nodes

```
ray start --address='MASTER_IP:6379'
```

4. Run Flask application

```
python app.py
```

8.4 Practical Performance Observations

Since the primary objective of the project was to demonstrate distributed task execution and parallel processing capabilities, the evaluation focused mainly on practical system behavior during execution within the laboratory environment.

Instead of collecting detailed benchmark timing values, the system was observed under multiple simultaneous player requests to analyze its response characteristics.

The following observations were obtained:

- Multiple player-related tasks executed simultaneously across worker nodes
- Task allocation occurred dynamically among available workers
- Worker nodes remained active during concurrent requests

- Response generation appeared more efficient compared to sequential execution
- System execution remained stable during repeated testing

The distributed implementation demonstrated effective workload distribution and improved utilization of computational resources.

8.5 Discussion

The proposed implementation showed the practical advantages of distributed computing in handling independent player-related computations.

In traditional sequential systems, tasks are executed one after another:

$$Total\ Processing\ Time = T_1 + T_2 + T_3 + \cdots + T_n$$

In distributed systems, multiple workers execute tasks simultaneously:

$$Total\ Processing\ Time \approx max(T_1, T_2, T_3, \ldots, T_n)$$

Although exact benchmark values were not collected during experimentation, practical execution demonstrated that task-level parallelism reduced waiting time and improved resource utilization.

Chapter 9

Implementation Details

This chapter describes the implementation details of the proposed Distributed Gaming Analytics System. The implementation integrates machine learning, statistical analysis, Flask-based backend services, and distributed computing using Ray.

9.1 Project Structure

The project is organized into multiple modules for better maintainability and separation of responsibilities.

File/Folder	Description
app.py	Flask backend and request handling
ray_tasks.py	Distributed task definitions
train_model.py	Machine learning model training
utils.py	Utility/helper functions
model/	Stores trained model files (.pkl)
data/	Dataset files
templates/	HTML pages for frontend
README.md	Project documentation

9.2 Backend Implementation

The backend was implemented using Flask.

Flask acts as the communication layer between the frontend and the distributed processing system.

Responsibilities of Flask include:

- Receiving user requests

- Processing input data
- Creating task objects
- Sending tasks to Ray workers
- Collecting worker outputs
- Returning final responses

Example implementation:

```
@app.route('/predict', methods=['POST'])
def predict():
    player_data = request.form
```

9.3 Ray Task Implementation

Distributed execution was implemented using Ray remote functions.

Example:

```
@ray.remote
def predict_player(player,target):
    prediction=model.predict(player)
    return prediction
```

The decorator:

`@ray.remote`

converts an ordinary Python function into a distributed task.

9.4 Task Scheduling

Player-related computations were distributed using future objects.

Implementation:

```
futures = [
predict_player.remote(player,target)
for player in players
]
```

Ray dynamically schedules these tasks among available worker nodes.

Advantages include:

- Automatic task allocation

- Efficient workload distribution
- Better resource utilization

9.5 Result Collection

Results generated by workers were collected using:

```
results = ray.get(futures)
```

The `ray.get()` function waits until all worker tasks complete execution.

9.6 Model Loading Strategy

Machine learning models were loaded locally inside worker nodes.

Implementation strategy:

```
model = load_model()
```

Advantages include:

- Reduced communication overhead
- Lower serialization cost
- Faster prediction execution

9.7 Implementation Flow

The complete implementation sequence can be summarized as:

1. User enters player statistics
2. Flask receives request
3. Flask generates tasks
4. Ray distributes tasks
5. Workers execute prediction and analysis
6. Results collected
7. Output displayed to users

Chapter 10

Work Distribution

The development of the proposed Distributed Gaming Analytics System involved dividing responsibilities among team members according to different project components.

Proper task allocation enabled efficient development and integration of various modules.

10.1 Responsibilities of Anindya Midhey

Anindya Midhey primarily contributed toward the analytical and user-interface components of the project. Responsibilities included machine learning model development, feature selection, statistical analysis design, performance score generation, interface design, and frontend integration.

10.2 Responsibilities of Ananyo Sen

Ananyo Sen primarily contributed toward backend implementation and distributed system development. Responsibilities included Ray cluster configuration, Flask backend development, distributed task scheduling, worker configuration, optimization implementation, and overall system integration.

10.3 Collaborative Activities

Both members jointly participated in architecture planning, system design discussions, experimental setup, testing, result analysis, and integration of individual components into the final system.

Chapter 11

Conclusion and Future Work

11.1 Conclusion

The proposed Distributed Gaming Analytics System successfully demonstrated the application of distributed computing for improving gaming analytics performance.

Traditional sequential approaches face challenges when handling multiple simultaneous requests due to increased waiting time and scalability limitations.

The use of Ray enabled efficient task-level parallelism by distributing independent player-related computations among worker nodes.

The proposed system integrated:

- Machine learning prediction
- Statistical analysis
- Flask backend services
- Distributed task scheduling
- Optimization techniques

Experimental observations showed that distributed execution reduced response time and improved resource utilization.

The implemented optimization techniques further reduced overhead and improved overall system efficiency.

The proposed architecture demonstrated improved scalability and efficient handling of multiple tasks.

11.2 Future Work

Although the proposed system achieved its objectives, several improvements may be implemented in future versions.

Possible future enhancements include:

- Deployment using cloud platforms
- Real-time streaming analytics
- Dynamic worker scaling
- Integration of advanced machine learning algorithms
- GPU-based distributed execution
- Live dashboard visualization
- Support for larger gaming datasets

Chapter 12

GitHub Repository

The complete implementation and source code for the project are available in the GitHub repository.

Repository Link:

`https://github.com/Ananyo-Sen-B2530066/distributed-gaming-analytics.git`

The repository includes:

- Complete source code
- Installation instructions
- Requirements file
- Worker node setup instructions
- Project documentation

Bibliography

[1] Moritz, P. et al., "Ray: A Distributed Framework for Emerging AI Applications," USENIX OSDI, 2018.

[2] Breiman, L., "Random Forests," Machine Learning Journal, 2001.

[3] Dean, J. and Ghemawat, S., "MapReduce: Simplified Data Processing on Large Clusters," OSDI, 2004.

[4] Flask Documentation: https://flask.palletsprojects.com

[5] Scikit-Learn Documentation: https://scikit-learn.org

[6] Ray Documentation: https://docs.ray.io